**OS & SP**

**Name :** D. Teja Sadwika **Roll No :** 2520030536

**SKILL – 12**

**1.Skill12.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   int pipefd[2];

   /* Create pipe */
   if (pipe(pipefd) == -1) {
      perror("pipe");
      return 1;
   }
```

```
printf("Pipe created successfully.\n");
fflush(stdout);

/* Create child process */
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    close(pipefd[0]);
    close(pipefd[1]);
    return 1;
}

/* Child process */
if (pid == 0) {
    close(pipefd[1]);
```

```
/* Redirect pipe input to stdin */
if (dup2(pipefd[0], STDIN_FILENO) == -1) {
    perror("dup2");
    close(pipefd[0]);
    _exit(1);
}

close(pipefd[0]);

printf("Child: Reading data from pipe...\n");
fflush(stdout);

char buffer[100];
ssize_t n = read(STDIN_FILENO, buffer,
                 sizeof(buffer) - 1);

if (n > 0) {
    buffer[n] = '\0';
```

```
        printf("Child received: %s\n", buffer);
    } else if (n == -1) {
        perror("read");
        _exit(1);
    }
    _exit(0);
}
/* Parent process */
close(pipefd[0]);
char message[] = "Hello from parent process!";
printf("Parent: Data written to pipe.\n");
if (write(pipefd[1], message, sizeof(message)) == -1)
    perror("write");
/* Close write end */
close(pipefd[1]);
/* Wait for child */
waitpid(pid, NULL, 0);
printf("Parent: Child process completed.\n");
```

```
    return 0;
}
```

**OUTPUT :**

